Task 4: Banking System

CodeAlpha C++ Internship | Marathi + English explanation

Program kay karto? (What it does)

• Ek simple banking system - customer banavto, account kholto, deposit/withdraw/transfer karto ani transaction history dakhavto.

Classes (OOP)

• Transaction: type, amount, balanceAfter, time, note - pratyek vyavahar cha record.
• Account: accountNumber, balance ani transaction history (vector). deposit(), withdraw(), showHistory() functions.
• Customer: id, name, phone ani tyache accounts (ek customer cha ekapeksha jast accounts hou shaktat).
• Bank: sarva customers manage karto, accounts shodhto, transfer logic handle karto.

Features

• Deposit: amount > 0 pahije.
• Withdraw: amount > 0 ani balance peksha jast nako, nahitar fail.
• Transfer: from account madhun withdraw, to account madhe deposit. Same account madhe transfer block.
• Recent transactions: shevtche 5 vyavahar date/time sobat.
• Account info: customer details ani balance.

Video madhe kay bolaych (Video tips)

• Pahile 4 classes cha relationship samjava (Bank -> Customer -> Account -> Transaction).
• Demo: 2 customers banva, deposit, withdraw, transfer kara, mag history dakhava.
• Insufficient balance cha error dakhava.
• Limitation sanga: data memory madhe aahe, program band kela ki jato. Future: file madhe save karne.

Kasa run karaycha (How to Run)

```
g++ -std=c++17 CodeAlpha_Banking_System.cpp -o bank
./bank            (Windows: bank.exe)
```

Full C++ Code

```
// CodeAlpha C++ Internship - Task 4: Banking System
#include <iostream>
#include <iomanip>
#include <vector>
#include <string>
#include <ctime>
using namespace std;

string currentTime() {
    time_t now = time(0);
    char buf[30];
    strftime(buf, sizeof(buf), "%d-%m-%Y %H:%M:%S", localtime(&now));
    return string(buf);
}

// ---------------- Transaction ----------------
class Transaction {
public:
    string type;      // Deposit, Withdraw, Transfer In, Transfer Out
    double amount;
    double balanceAfter;
    string time;
    string note;

    Transaction(string t, double a, double b, string n = "")
        : type(t), amount(a), balanceAfter(b), time(currentTime()), note(n) {}

    void display() const {
        cout << left << setw(21) << time << setw(14) << type
             << right << setw(10) << fixed << setprecision(2) << amount
             << setw(12) << balanceAfter << "  " << note << "\n";
    }
};

// ---------------- Account ----------------
class Account {
private:
    int accountNumber;
    double balance;
    vector<Transaction> history;

public:
    Account(int no, double initial = 0) : accountNumber(no), balance(initial) {
        if (initial > 0) history.push_back(Transaction("Deposit", initial, balance, "Opening balance"));
    }

    int getNumber() const { return accountNumber; }
    double getBalance() const { return balance; }

    bool deposit(double amt, string note = "") {
        if (amt <= 0) return false;
        balance += amt;
        history.push_back(Transaction(note.empty() ? "Deposit" : "Transfer In", amt, balance, note));
        return true;
    }

    bool withdraw(double amt, string note = "") {
        if (amt <= 0 || amt > balance) return false;
        balance -= amt;
        history.push_back(Transaction(note.empty() ? "Withdraw" : "Transfer Out", amt, balance, note));
        return true;
    }

    void showHistory(int last = 5) const {
        cout << "\nRecent Transactions (Account " << accountNumber << ")\n";
        if (history.empty()) { cout << "No transactions yet.\n"; return; }
        cout << left << setw(21) << "Date/Time" << setw(14) << "Type"
             << right << setw(10) << "Amount" << setw(12) << "Balance" << "  Note\n";
        cout << string(70, '-') << "\n";
        int start = (int)history.size() > last ? (int)history.size() - last : 0;
        for (int i = start; i < (int)history.size(); i++) history[i].display();
    }

    void showInfo() const {
        cout << "Account No : " << accountNumber << "\n";
        cout << "Balance    : " << fixed << setprecision(2) << balance << "\n";
    }
```

```
};

// ---------------- Customer ----------------
class Customer {
private:
    int customerId;
    string name;
    string phone;
    vector<Account> accounts;

public:
    Customer(int id, string n, string p) : customerId(id), name(n), phone(p) {}

    int getId() const { return customerId; }
    string getName() const { return name; }
    vector<Account> &getAccounts() { return accounts; }

    void addAccount(const Account &a) { accounts.push_back(a); }

    Account *findAccount(int no) {
        for (auto &a : accounts) if (a.getNumber() == no) return &a;
        return nullptr;
    }

    void showInfo() {
        cout << "\n===== Customer Details =====\n";
        cout << "ID    : " << customerId << "\n";
        cout << "Name  : " << name << "\n";
        cout << "Phone : " << phone << "\n";
        for (auto &a : accounts) { cout << "----------------------\n"; a.showInfo(); }
    }
};

// ---------------- Bank ----------------
class Bank {
private:
    vector<Customer> customers;
    int nextCustomerId = 1001;
    int nextAccountNo  = 50001;

public:
    Customer *findCustomer(int id) {
        for (auto &c : customers) if (c.getId() == id) return &c;
        return nullptr;
    }

    Account *findAccountAnywhere(int accNo) {
        for (auto &c : customers) {
            Account *a = c.findAccount(accNo);
            if (a) return a;
        }
        return nullptr;
    }

    void createCustomer() {
        string name, phone;
        double initial;
        cout << "Customer name: ";
        cin >> ws; getline(cin, name);
        cout << "Phone: ";
        cin >> phone;
        cout << "Opening deposit: ";
        cin >> initial;
        if (initial < 0) { cout << "Invalid amount.\n"; return; }

        Customer c(nextCustomerId, name, phone);
        c.addAccount(Account(nextAccountNo, initial));
        customers.push_back(c);
        cout << "Customer created! ID: " << nextCustomerId
             << " | Account No: " << nextAccountNo << "\n";
        nextCustomerId++; nextAccountNo++;
    }

    void openExtraAccount() {
        int id; double initial;
        cout << "Customer ID: "; cin >> id;
        Customer *c = findCustomer(id);
        if (!c) { cout << "Customer not found.\n"; return; }
        cout << "Opening deposit: "; cin >> initial;
```

```
        if (initial < 0) { cout << "Invalid amount.\n"; return; }
        c->addAccount(Account(nextAccountNo, initial));
        cout << "New account opened: " << nextAccountNo << "\n";
        nextAccountNo++;
    }

    void deposit() {
        int no; double amt;
        cout << "Account No: "; cin >> no;
        Account *a = findAccountAnywhere(no);
        if (!a) { cout << "Account not found.\n"; return; }
        cout << "Amount: "; cin >> amt;
        if (a->deposit(amt)) cout << "Deposit successful. New balance: " << fixed << setprecision(2) << a->getBalance() <<
        else cout << "Invalid amount.\n";
    }

    void withdraw() {
        int no; double amt;
        cout << "Account No: "; cin >> no;
        Account *a = findAccountAnywhere(no);
        if (!a) { cout << "Account not found.\n"; return; }
        cout << "Amount: "; cin >> amt;
        if (a->withdraw(amt)) cout << "Withdrawal successful. New balance: " << fixed << setprecision(2) << a->getBalance(
        else cout << "Failed: invalid amount or insufficient balance.\n";
    }

    void transfer() {
        int from, to; double amt;
        cout << "From Account No: "; cin >> from;
        cout << "To Account No: ";   cin >> to;
        if (from == to) { cout << "Cannot transfer to same account.\n"; return; }
        Account *a = findAccountAnywhere(from);
        Account *b = findAccountAnywhere(to);
        if (!a || !b) { cout << "One or both accounts not found.\n"; return; }
        cout << "Amount: "; cin >> amt;
        if (a->withdraw(amt, "To " + to_string(to))) {
            b->deposit(amt, "From " + to_string(from));
            cout << "Transfer successful.\n";
        } else cout << "Transfer failed: invalid amount or insufficient balance.\n";
    }

    void viewTransactions() {
        int no;
        cout << "Account No: "; cin >> no;
        Account *a = findAccountAnywhere(no);
        if (!a) { cout << "Account not found.\n"; return; }
        a->showHistory(5);
    }

    void viewCustomer() {
        int id;
        cout << "Customer ID: "; cin >> id;
        Customer *c = findCustomer(id);
        if (!c) { cout << "Customer not found.\n"; return; }
        c->showInfo();
    }
};

int main() {
    Bank bank;
    int choice;
    do {
        cout << "\n=========== BANKING SYSTEM ===========\n";
        cout << "1. Create Customer + Account\n";
        cout << "2. Open Extra Account\n";
        cout << "3. Deposit\n";
        cout << "4. Withdraw\n";
        cout << "5. Transfer Funds\n";
        cout << "6. View Recent Transactions\n";
        cout << "7. View Customer / Account Info\n";
        cout << "0. Exit\n";
        cout << "Choice: ";
        if (!(cin >> choice)) { cin.clear(); cin.ignore(1000, '\n'); choice = -1; }
        switch (choice) {
            case 1: bank.createCustomer();     break;
            case 2: bank.openExtraAccount();   break;
            case 3: bank.deposit();            break;
            case 4: bank.withdraw();           break;
            case 5: bank.transfer();           break;
```

```
            case 6: bank.viewTransactions();   break;
            case 7: bank.viewCustomer();       break;
            case 0: cout << "Thank you for banking with us!\n"; break;
            default: cout << "Invalid choice.\n";
        }
    } while (choice != 0);
    return 0;
}
```

Sample Output

```
Recent Transactions (Account 50001)
Date/Time            Type              Amount     Balance  Note
------------------------------------------------------------------------
29-10-2026 15:53:28  Deposit          5000.00     5000.00  Opening balance
29-10-2026 15:53:28  Deposit           500.00     5500.00
29-10-2026 15:53:28  Withdraw          200.00     5300.00
29-10-2026 15:53:28  Transfer Out     1500.00     3800.00  To 50002
```